

UNIVERSIDAD DEL CEMA

Buenos Aires, Argentina

El rol del ingeniero informático en los próximos diez años: de la ejecución técnica a la supervisión crítica de sistemas automatizados

Trabajo Práctico Final

Carrera: Ingeniería Informática (3.er año)

Materia: Lenguajes Formales y Teoría de la Computación

Profesor: Mario Moreno

Integrantes: Martín Ezequiel Pulitano y Nicolás Silva

Junio de 2026

Resumen

La irrupción de la inteligencia artificial generativa y de los agentes de programación autónomos ha reabierto una pregunta tan vieja como la propia disciplina: ¿seguirá existiendo el ingeniero informático? Este trabajo sostiene que, en la próxima década, el ingeniero no será reemplazado por la tecnología, sino que su rol se transformará: dejará de centrarse en la ejecución técnica para enfocarse en la especificación, la integración, la validación y la supervisión crítica de sistemas automatizados. El argumento se construye sobre tres pilares: la evidencia histórica de que cada salto de abstracción elevó el nivel del trabajo en lugar de eliminarlo; la evidencia empírica reciente (2021-2026) sobre el alcance y los límites medidos de la IA en el desarrollo de software; y, como columna vertebral teórica, los límites fundamentales de la computación —el problema de la detención, el teorema de Rice, la cuestión P vs. NP y la jerarquía de Chomsky— que garantizan la existencia de un núcleo irreductiblemente humano de juicio. La conclusión es necesariamente una estimación: el ingeniero del futuro ascenderá en la escalera de abstracción hacia tareas que la teoría de la computación demuestra que ninguna máquina puede automatizar por completo.

Palabras clave: ingeniería de software, inteligencia artificial, indecidibilidad, problema de la detención, P vs. NP, automatización.

I. Introducción

Cada vez que la tecnología avanzó, el rol del ingeniero informático cambió con ella: los lenguajes de alto nivel, los entornos integrados de desarrollo o la nube transformaron su trabajo diario sin hacer desaparecer la profesión. Hoy, la inteligencia artificial generativa —en particular los modelos grandes de lenguaje (LLM) y los agentes de codificación autónomos— plantea el desafío más radical hasta la fecha, al punto de que destacados referentes pronostican que la mayor parte del código será escrita por máquinas en pocos años.

Este trabajo parte de la siguiente hipótesis: en los próximos diez años, el rol del ingeniero dejará de centrarse en la ejecución técnica para enfocarse en la toma de decisiones, la integración de tecnologías y la supervisión crítica de sistemas automatizados. No será reemplazado, sino que se volverá menos operativo y más estratégico, con mayor responsabilidad en diseñar, validar y controlar sistemas complejos.

Para sostenerla se recorre la historia como sucesión de abstracciones (II), el estado actual de la IA (III) y los límites computacionales que explican *por qué* persiste un rol humano (IV), antes de caracterizar el nuevo rol, las visiones de expertos y las limitaciones (V a VII).

II. La escalera de abstracción: una historia de transformaciones, no de extinciones

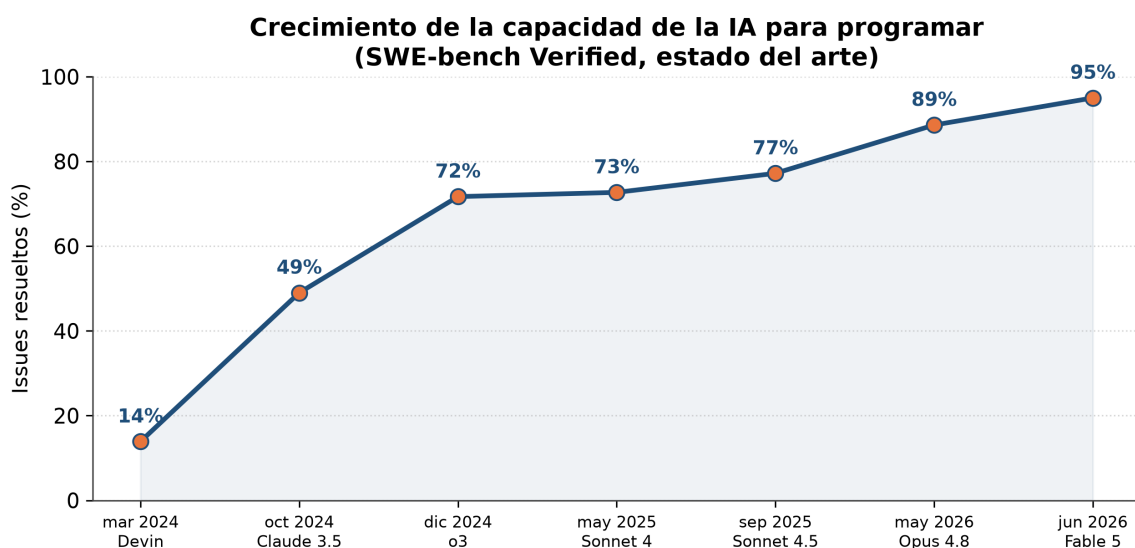
La historia del software puede leerse como el ascenso por una "escalera de abstracción" en la que cada peldaño automatizó el trabajo del anterior y, lejos de suprimir al ingeniero, lo elevó conceptualmente. Al principio, programar era escribir código máquina y ensamblador; el salto decisivo llegó con los lenguajes de alto nivel: FORTRAN (Backus, IBM, 1957) permitió expresar algoritmos en notación matemática y delegó el código de bajo nivel en un compilador; COBOL, inspirado en Grace Hopper, hizo lo propio para los negocios.

Paradójicamente, esa creciente potencia trajo una crisis: en la Conferencia de Ingeniería de Software de la OTAN (Garmisch, 1968) se acuñó la expresión "crisis del software" para describir la incapacidad de gestionar la complejidad de sistemas cada vez más grandes (Naur y Randell, 1969). La respuesta no fue menos ingeniería, sino más: ante un problema vuelto "igualmente gigantesco" por las máquinas (Dijkstra, 1972), nacieron la programación estructurada, las metodologías y la disciplina misma de la ingeniería de software.

Los peldaños siguientes confirman el patrón: los IDE comprimieron el ciclo de edición, compilación y prueba; Git (2005) y Stack Overflow (2008) compartieron el conocimiento; la nube (Amazon Web Services, 2006) abstrajo la infraestructura y DevOps (2009) automatizó el despliegue; y en 2021 GitHub Copilot introdujo el primer "programador en par". En cada caso la herramienta automatizó lo rutinario, pero el ingeniero permaneció y ascendió hacia el diseño de arquitecturas distribuidas. El anuncio de su desaparición tampoco es nuevo —ya se proclamó con COBOL, los lenguajes de cuarta generación y el "sin código"—; la tesis del "fin de la programación" (Welsh, 2023) es la más reciente de una larga serie de profecías incumplidas.

III. El presente: la IA agéntica en el desarrollo de software (2021-2026)

El ritmo del cambio es difícil de exagerar. En SWE-bench Verified, que mide la capacidad de la IA para resolver *issues* reales de software, el estado del arte pasó de ~14 % a comienzos de 2024 a cerca del 95 % a mediados de 2026 (Anthropic, 2026): de uno de cada siete casos a más de nueve de cada diez en poco más de dos años. La adopción acompaña: GitHub Copilot superó los veinte millones de usuarios, Sundar Pichai cifró en 2024 el código generado por IA en Google en "más de una cuarta parte" y Satya Nadella lo situó entre el 20 % y el 30 % en Microsoft (Zeff, 2025); la encuesta de Stack Overflow (2025) lo confirma: el 84 % de los desarrolladores usa o planea usar IA.

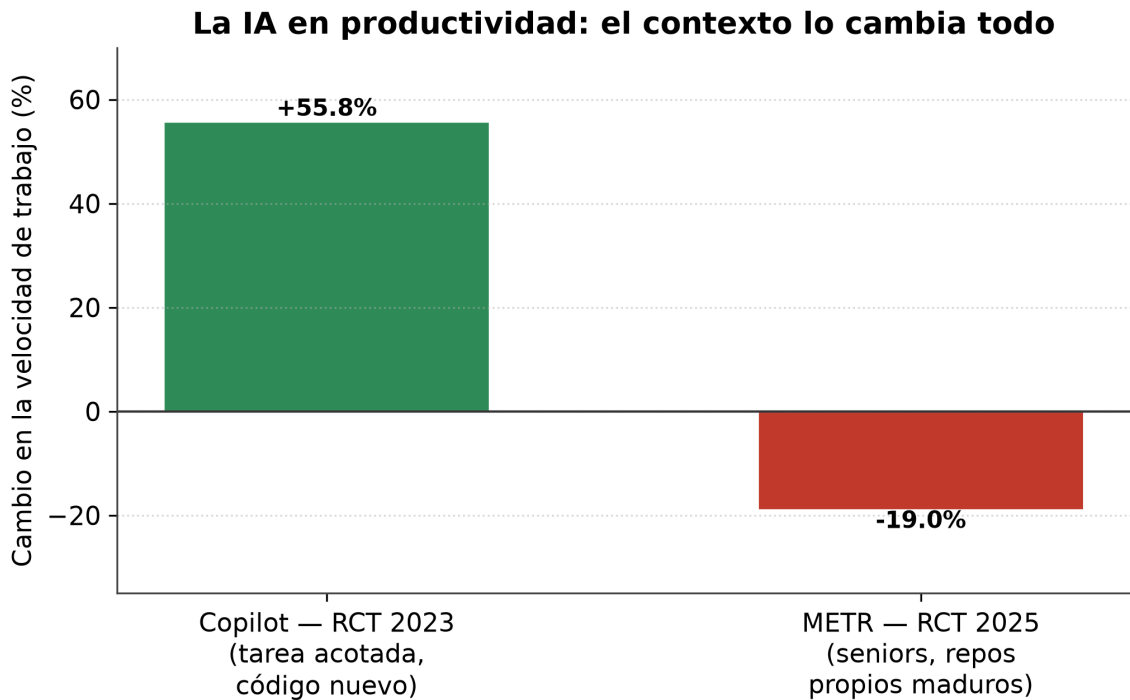


Fuente: SWE-bench Verified (Anthropic, OpenAI). Hacia 2026 el benchmark se satura (Fable 5 $\approx$ 95 %). El punto de mar. 2024 (Devin) es sobre otro subconjunto.

Figura 1. La capacidad de la IA para resolver issues reales de software pasó de ~14 % (inicios de 2024) a ~95 % (mediados de 2026) en SWE-bench Verified. Fuente: elaboración propia a partir de Anthropic (2026) y datos públicos de SWE-bench.

Conviene, no obstante, leer estos números con cautela: un benchmark no es el mundo real. El propio equipo de SWE-bench Verified advirtió que el conjunto se saturó y "ya no mide las capacidades de codificación de frontera", y la distancia con la práctica quedó expuesta cuando un grupo independiente probó al agente Devin —presentado en 2024 como "el primer ingeniero de software de IA"— en tareas reales: completó con éxito solo 3 de 20 (Husain et al., 2025). Resolver un *issue* aislado en un repositorio conocido es muy distinto de operar dentro de un sistema de producción, con su contexto y sus consecuencias.

Esa misma brecha reaparece al medir la productividad, donde la evidencia es más matizada de lo que sugiere el entusiasmo. Un experimento controlado de GitHub mostró que con Copilot una tarea acotada —programar un servidor HTTP— se completaba un 55,8 % más rápido (Peng et al., 2023), y un experimento de campo con 4.867 desarrolladores halló un aumento del 26 % en las tareas completadas (Cui et al., 2025). Pero ese último reveló un patrón decisivo: la mejora se concentraba en los programadores junior y casi se desvanecía entre los seniors. El contrapunto más nítido lo aportó un ensayo aleatorizado independiente de METR (2025): desarrolladores *experimentados* en sus propios repositorios maduros tardaron un 19 % *más* con IA, pese a que *creían* haberse acelerado un 20 %. Lejos de contradecirse, los estudios convergen en una lección: la IA acelera lo rutinario y lo desconocido para quien empieza, pero rinde poco —o estorba— en el trabajo experto sobre sistemas maduros. La brecha entre productividad percibida y real es, en sí misma, reveladora.



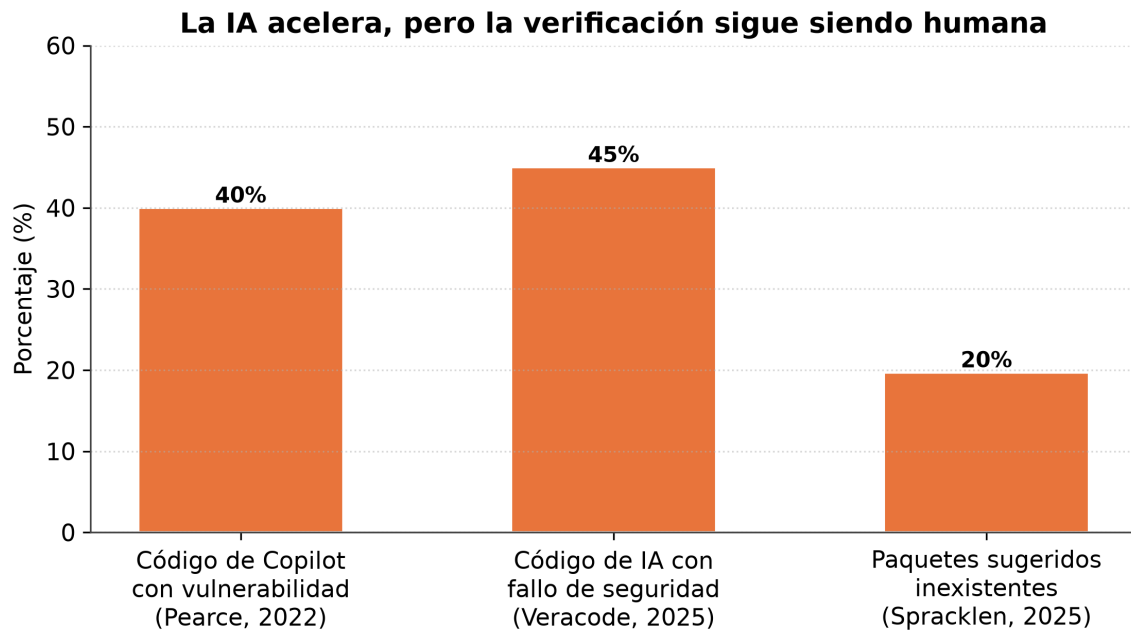
Fuente: Peng et al. (2023); METR (2025). Estudios no comparables directamente (tarea y experiencia distintas).

Figura 2. La misma tecnología acelera o frena según el contexto: +55,8 % en una tarea acotada (Peng et al., 2023) frente a -19 % en desarrolladores expertos sobre repositorios propios maduros (METR, 2025).

A esa ambivalencia se añade un efecto sobre la confianza y la formación. Según Stack Overflow (2025), la confianza en la exactitud de las salidas de IA cayó a alrededor de un tercio, y el 45 % afirma que depurar código de IA lleva *más* tiempo que escribirlo. Y el problema trasciende la productividad: un ensayo de Anthropic con ingenieros mayormente junior observó que quienes trabajaban con IA puntuaban sensiblemente menos en una prueba posterior sin ella —sobre todo en depuración—, lo que sugiere que delegar demasiado pronto erosiona el aprendizaje del juicio que la supervisión exigirá (Shen y Tamkin, 2026).

A esta tensión se suman la calidad y la seguridad. Addy Osmani (Google) describió el "problema del 70 %": la IA produce con rapidez el 70 % de una solución, pero el 30 % final —casos límite, depuración, integración con producción, mantenibilidad— sigue exigiendo juicio experto (Osmani, 2024). Un estudio de Stanford halló que, con un asistente de IA, los programadores escribían código *menos* seguro y a la vez confiaban *más* en él —una falsa confianza peligrosa (Perry et al., 2023)—. Las evaluaciones de corrección apuntan en el mismo sentido: al endurecer los tests, la tasa de acierto cae entre 19 y 29 puntos (Liu et al., 2023), lo que revela que "pasar los tests" no equivale a satisfacer la especificación. En seguridad, cerca del 40 % de las sugerencias de Copilot en escenarios sensibles incluían vulnerabilidades conocidas (Pearce et al., 2022), y un informe de 2025 halló código vulnerable en el 45 % de los casos, sin que los modelos más nuevos fueran más seguros (Veracode, 2025). Se suma un riesgo inédito: cerca del 20 % de los paquetes que los modelos recomiendan importar no existen, lo que habilita el *slopsquatting* —registrarlos con código malicioso (Spracklen et al., 2025)—. La

IA, en suma, acelera la producción pero no garantiza la corrección: traslada el cuello de botella desde la escritura del código hacia su verificación.



Fuente: Pearce et al. (2022); Veracode (2025); Spracklen et al. (2025).

Figura 3. La IA traslada el cuello de botella hacia la verificación humana: proporción de salidas con vulnerabilidades o paquetes inexistentes. Fuente: Pearce et al. (2022); Veracode (2025); Spracklen et al. (2025).

IV. El marco teórico: los límites computacionales que protegen el rol del ingeniero

¿Por qué, pese a estos avances, el ingeniero no desaparecerá? La respuesta más profunda no es económica ni sociológica, sino computacional: existen barreras matemáticas, demostradas y permanentes, que ninguna IA —en última instancia, un proceso computable— puede sortear.

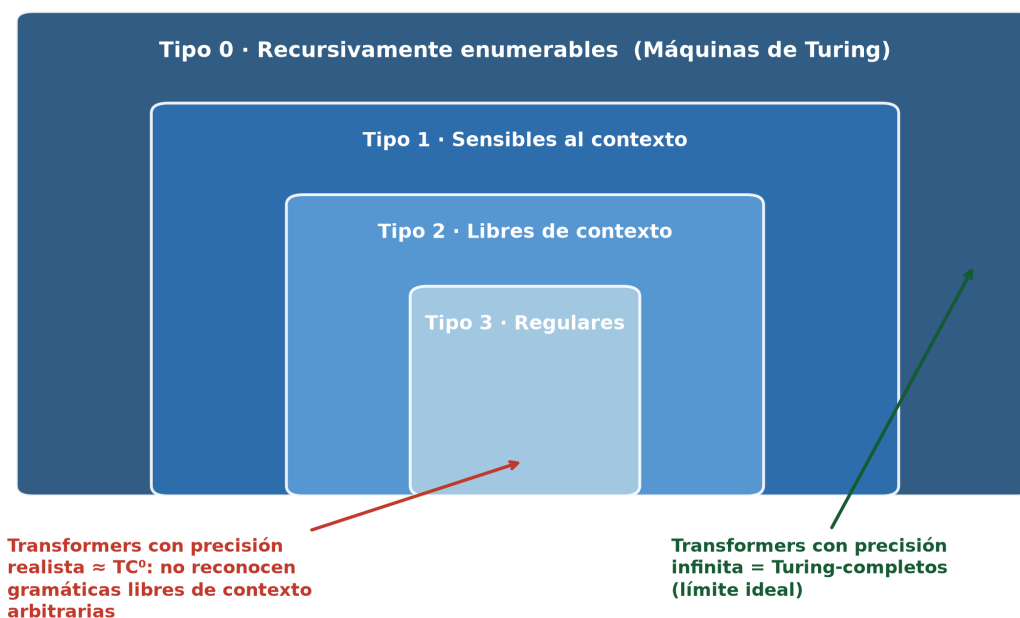
La primera es la **indecidibilidad**. En 1936, Turing demostró que el problema de la detención es indecible: ningún algoritmo puede determinar, para todo programa y entrada, si terminará o correrá indefinidamente (Turing, 1936). El teorema de Rice (1953) lo generaliza: *toda* propiedad semántica no trivial de un programa —como "¿está libre de cierto error?" o "¿cumple su especificación?"— es indecible. Por tanto, ninguna IA puede verificar de forma general y automática si un programa arbitrario satisface su especificación: la verificación seguirá exigiendo juicio humano y herramientas que solo funcionan restringiendo el problema o renunciando a la completitud.

La segunda barrera es la **intratabilidad**. El problema P vs. NP, uno de los siete Problemas del Milenio, pregunta si todo problema cuya solución se verifica rápido puede también resolverse rápido (Cook, 1971). Bajo la hipótesis aceptada de que $P \neq NP$, numerosos problemas centrales de la ingeniería (planificación, ruteo, asignación de recursos) son NP-difíciles y no admiten soluciones eficientes en el peor caso. Es un límite matemático: ninguna IA lo derogará. El ingeniero seguirá eligiendo heurísticas y compromisos; decidir *qué*

sacrificar —optimalidad, tiempo o recursos— es un acto de juicio, no un cálculo automatizable.

La tercera barrera concierne al lenguaje. La jerarquía de Chomsky (1956) ordena los lenguajes formales según su poder generativo, y los de programación pertenecen, en su sintaxis, a los libres de contexto: formales, precisos, analizables sin ambigüedad. El lenguaje natural —y las instrucciones que un humano da a un LLM— es, en cambio, ambiguo y dependiente del contexto. El rol perdurable del ingeniero es el de traductor: convertir la intención humana, informal e incompleta, en una especificación formal ejecutable. No es una analogía: los propios *transformers* están acotados por la jerarquía de Chomsky y, bajo supuestos realistas de precisión, pertenecen a la clase TC^0 , incapaz de reconocer gramáticas libres de contexto arbitrarias (Delétang et al., 2023; Merrill y Sabharwal, 2023).

Los transformers, dentro de la jerarquía de Chomsky



Fuente: Chomsky (1956); Delétang et al. (2023); Merrill & Sabharwal (2023); Pérez et al. (2021).

Figura 4. Los transformers, situados dentro de la jerarquía de Chomsky: con precisión realista equivalen a la clase de complejidad TC^0 . Fuente: elaboración propia a partir de Chomsky (1956), Delétang et al. (2023) y Merrill y Sabharwal (2023).

Estas barreras alcanzan a la propia IA. Mediante diagonalización apoyada en la indecidibilidad de la detención, se demuestra que la "alucinación" es innata a los LLM: ningún modelo puede aprender todas las funciones computables, por lo que producirá errores que no detecta por sí mismo (Xu et al., 2024). Un LLM no puede autoverificar su salida; necesita un agente externo —el ingeniero— que la contraste con la realidad y asuma la responsabilidad.

Finalmente, Brooks (1987) distinguió la complejidad **accidental** —la del proceso de construcción, que las herramientas reducen— de la **esencial**, la dificultad conceptual inherente al problema. "La parte más difícil de construir un sistema de software es decidir con precisión

qué construir": la IA ataca lo accidental, pero decidir qué construir, para quién y con qué compromisos éticos es una tarea humana cargada de valores. En la misma línea, Naur (1985) sostuvo que el verdadero producto de programar es una *teoría* del problema que reside en la mente del equipo. Ahí reside el corazón irreducible de la ingeniería: no en *toda* la programación —la verificación acotada y de bajo riesgo se automatiza— sino en su franja crítica, donde la indecidibilidad se cruza con sistemas de alto riesgo y con una responsabilidad que ninguna norma legal admite delegar en una máquina.

V. El nuevo rol: del ejecutor al supervisor crítico

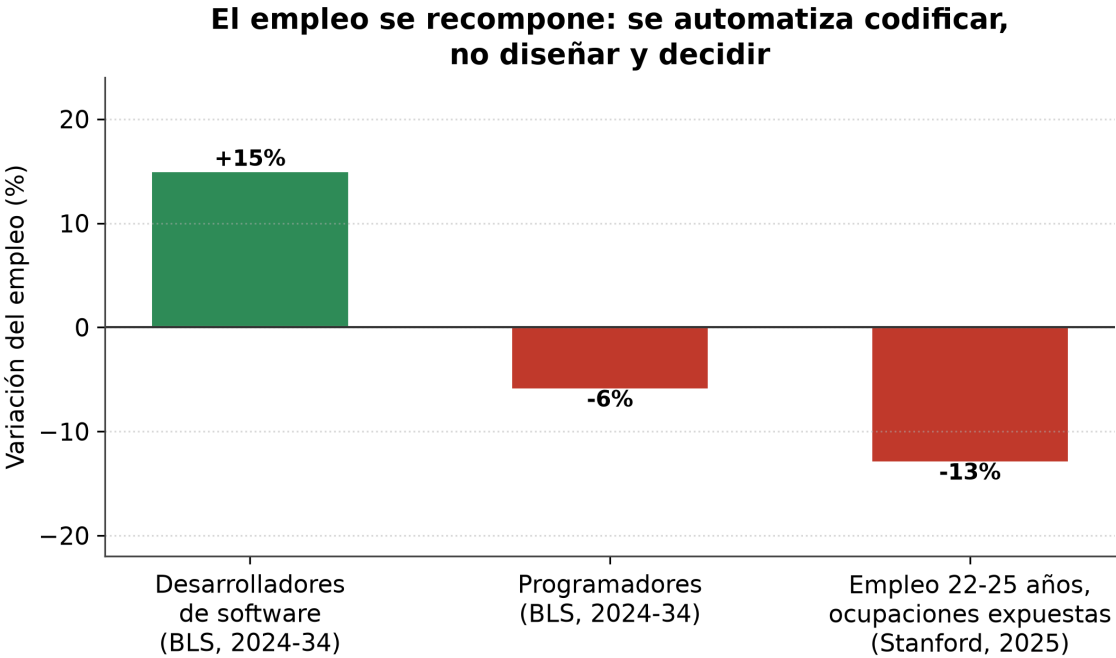
Si la teoría garantiza un núcleo humano, la economía explica por qué podría ampliarse. Vale la "paradoja de Jevons": cuando una tecnología abarata un recurso, su consumo total tiende a aumentar. Los cajeros automáticos son el caso clásico: abarataron la sucursal, los bancos abrieron más y el empleo creció (Bessen, 2015). En el software la lógica es análoga: al abaratare la producción, la demanda —y la de quienes lo diseñan y controlan— podría crecer (Acemoglu y Restrepo, 2019; Autor, 2015).

El nuevo rol es un peldaño más en la escalera de abstracción. El ingeniero del futuro escribirá menos código línea por línea y dedicará su tiempo a: **especificar** qué debe hacer el sistema; **diseñar** arquitecturas e integrar componentes heterogéneos, incluidos los modelos de IA; **validar y verificar** el código generado, sabiendo —por Rice y Turing— que esa validación no puede delegarse del todo; **supervisar** sistemas autónomos (seguridad, sesgo, impacto); y **decidir** bajo restricciones de intratabilidad y de valores. Es el "30 % difícil" de Osmani vuelto centro de gravedad de la profesión. Las proyecciones lo confirman: cerca del 40 % de las habilidades cambiarán hacia 2030 (World Economic Forum, 2025), y marcos como el SWEBOK (Washizaki, 2024) ya definen la ingeniería muy por encima de la codificación.

Proyectada a 2035, esa transformación apunta a un ingeniero que orchestra flotas de agentes más que a uno que teclea funciones: define la intención, delega la ejecución y responde por el resultado. Kent Beck lo expresó al constatar que "el 90 % de mis habilidades hoy valen 0 dólares", mientras que el 10 % restante —visión, diseño y control de la complejidad— pasó a valer mil veces más (Beck, 2023). Y Martin Fowler advierte que los LLM introducen una abstracción *no determinista*, sin la reproducibilidad de un compilador, comparable al paso del ensamblador a los lenguajes de alto nivel (Fowler, 2025). El nuevo peldaño no elimina el juicio: lo vuelve más abstracto y decisivo.

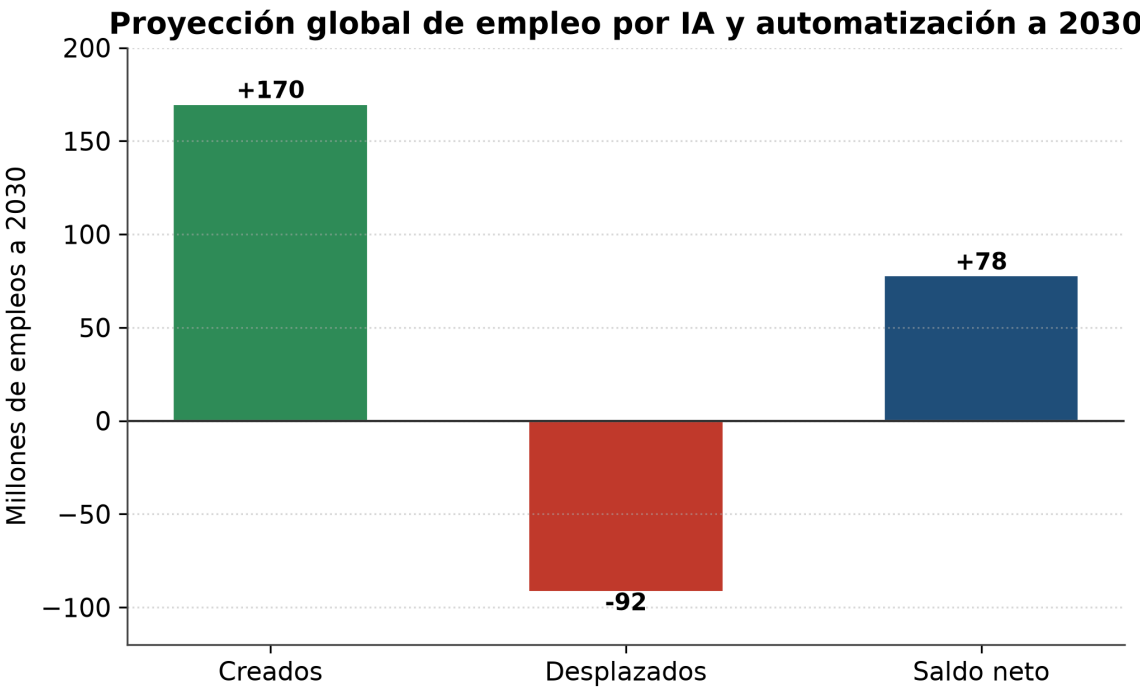
Los datos laborales, sin embargo, advierten que la transición no será indolora ni uniforme. La Oficina de Estadísticas Laborales de EE. UU. proyecta un alza cercana al 15 % en "desarrolladores de software" (2024-2034), mientras "programadores" se contraería un 6 % (Bureau of Labor Statistics, 2025): se automatiza codificar, no diseñar y decidir. El AI Index de Stanford afinó el dato: hacia mediados de 2026, el empleo de desarrolladores de 22-25 años había caído cerca del 20 % desde 2024 —en un año con 142.000 despidos tecnológicos—

mientras el de los mayores de 30 crecía en las mismas empresas (Brynjolfsson et al., 2025; Stanford HAI, 2026). La "puerta de entrada" junior se estrecha, y su cierre no es solo laboral sino *cognitivo*: si la IA absorbe las tareas donde se forma el criterio, faltarán los expertos capaces de auditarla. Y aun conservando el empleo, el ingeniero afronta un riesgo distributivo: que el valor migre hacia quienes proveen las herramientas de IA.



Fuente: U.S. Bureau of Labor Statistics (2025); Brynjolfsson, Chandar & Chen (2025).

Figura 5. El empleo se recompone: crece "desarrollador de software" mientras se contraen "programador" y el empleo joven en ocupaciones expuestas a la IA. Fuente: Bureau of Labor Statistics (2025); Brynjolfsson et al. (2025).



Fuente: World Economic Forum, Future of Jobs Report 2025.

Figura 6. Proyección global a 2030: 170 millones de empleos creados, 92 millones desplazados, saldo neto +78 millones.
Fuente: World Economic Forum (2025).

VI. Visiones de los expertos: optimismo, cautela y un consenso implícito

Como toda proyección, depende de quienes la construyen, cuyas opiniones, aunque dispares, convergen en un punto. Dario Amodei (Anthropic) pronosticó en 2025 que la IA escribiría el 90 % del código en meses y "esencialmente todo" en un año, pero aclaró que "el programador todavía debe especificar (...) las decisiones generales de diseño" (Council on Foreign Relations, 2025). A mediados de 2026 ese pronóstico se cumplió *de puertas adentro* —en la propia Anthropic casi todo el código lo escribe la IA (Krieger, 2026)—, pero no a escala industrial, donde sigue muy por debajo. Andrej Karpathy describió el "Software 3.0" y popularizó el "vibe coding" (Karpathy, 2025).

No todas las voces comparten ese optimismo. Grady Booch, coautor de UML, replicó que tales pronósticos revelan "una incomprensión fundamental de lo que es la ingeniería de software" (Booch, 2026). Bjarne Stroustrup, creador de C++, advirtió que el riesgo no es tanto el reemplazo como que los programadores "pierdan la capacidad de detectar problemas por estar tan acostumbrados a que se los resuelvan" (Stroustrup, 2025), un peligro que el apartado IV fundamenta.

Existe, además, un bando que anticipa no una transformación sino un reemplazo. Amodei advirtió que la IA podría eliminar la mitad de los empleos administrativos de nivel inicial en uno a cinco años y elevar el desempleo al 10-20 % (VandeHei y Allen, 2025); Mark Zuckerberg proyectó que cubriría el trabajo de los ingenieros de nivel medio de Meta, y Salesforce congeló la contratación de ingenieros en 2025 tras reportar un 30 % más de productividad (The San Francisco Standard, 2025). En clave académica, Frey y Osborne (2017) estimaron en un 47 % el empleo estadounidense en alto riesgo. No conviene minimizarlas, pero admiten reparos: aquel estudio medía ocupaciones enteras, no tareas, y trabajos posteriores lo corrigieron a la baja; las predicciones de directivos cargan un interés comercial evidente —Salesforce sigue empleando unos 15.000 ingenieros y Benioff admite que la IA aún no los sustituye—; y el desplazamiento golpea la tarea rutinaria de codificar, no el juicio de especificar, validar y decidir que el apartado IV vuelve irreductible.

Sam Altman (OpenAI) ofrece la síntesis más equilibrada: "cada ingeniero de software hará, durante un buen tiempo, muchísimo más" (Thompson, 2025), y "los expertos seguirán siendo mucho mejores que los novatos, si adoptan las nuevas herramientas" (Altman, 2025). La IA no anula al ingeniero: amplifica al que sabe usarla. Incluso las voces más optimistas reservan para el humano las tareas de especificar, decidir y juzgar —las mismas que la teoría de la computación demuestra que no pueden automatizarse.

VII. Limitaciones del análisis

Toda predicción tecnológica a diez años es incierta: la conclusión es una estimación fundada, no una certeza. Buena parte de la evidencia proviene de estudios recientes —algunos aún *working papers* sin revisión por pares— y de declaraciones de ejecutivos con intereses comerciales, lo que obliga a leerlas con cautela. La causalidad de los fenómenos laborales es además discutible: la contracción del empleo junior podría obedecer tanto a la IA como al fin del crédito barato y a la sobrecontratación previa. La base sólida es el marco teórico: los resultados de Turing, Rice, Cook y Chomsky no caducan con la próxima versión de un modelo, y son el fundamento más firme de la tesis aquí defendida.

VIII. Conclusión

El recorrido confirma, con matices, la hipótesis inicial. En la próxima década el ingeniero informático no será reemplazado por la IA, pero su rol se transformará: el centro de gravedad de la profesión pasará de la ejecución técnica —escribir código línea por línea— a la especificación, el diseño, la integración y, sobre todo, la validación y supervisión crítica de sistemas cada vez más autónomos. La historia lo respalda, pues cada abstracción previa elevó al ingeniero en vez de eliminarlo; la evidencia reciente lo matiza, pues la IA acelera la producción pero degrada la garantía de corrección; y la teoría de la computación lo fundamenta: la verificación universal (Rice y Turing), la resolución eficiente de todo problema (P vs. NP) y la traducción de la intención ambigua al lenguaje formal (Chomsky) no son tareas que una máquina pueda asumir por completo.

La paradoja final es esperanzadora: cuanto más poderosa se vuelve la automatización, más valioso es el juicio humano que decide qué automatizar, cómo validarlo y para qué fin. El ingeniero del futuro será menos un escriba y más un arquitecto, un auditor y un responsable último de sistemas que ni él ni la máquina comprenden del todo. La teoría de la computación, lejos de anunciar el fin de la profesión, traza el contorno de su porvenir.

Bibliografía

- Acemoglu, D., & Restrepo, P. (2019). Automation and new tasks: How technology displaces and reinstates labor. *Journal of Economic Perspectives*, 33(2), 3-30. <https://doi.org/10.1257/jep.33.2.3>
- Altman, S. (2025, 10 de junio). *The gentle singularity*. <https://blog.samaltman.com/the-gentle-singularity>
- Anthropic. (2026, 9 de junio). *Claude Fable 5*. <https://www.anthropic.com/news/claude-fable-5>
- Autor, D. H. (2015). Why are there still so many jobs? The history and future of workplace automation. *Journal of Economic Perspectives*, 29(3), 3-30. <https://doi.org/10.1257/jep.29.3.3>
- Beck, K. (2023, 19 de abril). *90% of my skills are now worth \$0*. Tidy First? <https://newsletter.kentbeck.com/p/90-of-my-skills-are-now-worth-0>
- Bessen, J. (2015). Toil and technology. *Finance & Development*, 52(1), 16-19. Fondo Monetario Internacional. <https://www.imf.org/external/pubs/ft/fandd/2015/03/bessen.htm>
- Booch, G. (2026, 4 de febrero). *The third golden age of software engineering — Thanks to AI, with Grady Booch* [Episodio de podcast]. The Pragmatic Engineer (G. Orosz, entrevistador).

- <https://newsletter.pragmaticengineer.com/p/the-third-golden-age-of-software>
- Brooks, F. P. (1987). No silver bullet: Essence and accidents of software engineering. *Computer*, 20(4), 10-19. <https://doi.org/10.1109/MC.1987.1663532>
- Brynjolfsson, E., Chandar, B., & Chen, R. (2025). *Canaries in the coal mine? Six facts about the recent employment effects of artificial intelligence*. Stanford Digital Economy Lab. <https://digitaleconomy.stanford.edu/publications/canaries-in-the-coal-mine/>
- Bureau of Labor Statistics. (2025). *Software developers, quality assurance analysts, and testers*. Occupational Outlook Handbook. U.S. Department of Labor. <https://www.bls.gov/ooh/computer-and-information-technology/software-developers.htm>
- Chomsky, N. (1956). Three models for the description of language. *IRE Transactions on Information Theory*, 2(3), 113-124. <https://doi.org/10.1109/TIT.1956.1056813>
- Cook, S. A. (1971). The complexity of theorem-proving procedures. En *Proceedings of the Third Annual ACM Symposium on Theory of Computing (STOC '71)* (pp. 151-158). Association for Computing Machinery. <https://doi.org/10.1145/800157.805047>
- Council on Foreign Relations. (2025, 10 de marzo). *CEO speaker series with Dario Amodei of Anthropic* [Transcripción del evento]. <https://www.cfr.org/event/ceo-speaker-series-dario-amodei-anthropic>
- Cui, Z. K., Demirer, M., Jaffe, S., Musolff, L., Peng, S., & Salz, T. (2025). *The effects of generative AI on high-skilled work: Evidence from three field experiments with software developers* [Documento de trabajo]. Massachusetts Institute of Technology. https://economics.mit.edu/sites/default/files/inline-files/draft_copilot_experiments.pdf
- Delétang, G., Ruoss, A., Grau-Moya, J., Genewein, T., Wenliang, L. K., Catt, E., Cundy, C., Hutter, M., Legg, S., Veness, J., & Ortega, P. A. (2023). *Neural networks and the Chomsky hierarchy* [Ponencia]. International Conference on Learning Representations (ICLR 2023). <https://doi.org/10.48550/arXiv.2207.02098>
- Dijkstra, E. W. (1972). The humble programmer. *Communications of the ACM*, 15(10), 859-866. <https://doi.org/10.1145/355604.361591>
- Fowler, M. (2025, 24 de junio). *LLMs bring new nature of abstraction*. martinofowler.com. <https://martinfofowler.com/articles/2025-nature-abstraction.html>
- Frey, C. B., & Osborne, M. A. (2017). The future of employment: How susceptible are jobs to computerisation? *Technological Forecasting and Social Change*, 114, 254-280. <https://doi.org/10.1016/j.techfore.2016.08.019>
- Husain, H., Flath, I., & Whitaker, J. (2025, 8 de enero). *Thoughts on a month with Devin*. Answer.AI. <https://www.answer.ai/posts/2025-01-08-devin.html>
- Karpathy, A. (2025, 17 de junio). *Software is changing (again)* [Video]. Y Combinator AI Startup School. YouTube. <https://www.youtube.com/watch?v=LCEmiRjPEtQ>
- Krieger, M. (2026). *Dario Amodei had predicted 90% of code would be written by AI, but now at Anthropic it's effectively 100%*. OfficeChai. <https://officechai.com/ai/dario-amodei-had-predicted-90-of-code-would-be-written-by-ai-but-now-at-anthropic-its-effectively-100-anthropic-cpo/>
- Liu, J., Xia, C. S., Wang, Y., & Zhang, L. (2023). Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation. En *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)* (pp. 21558-21572). <https://doi.org/10.48550/arXiv.2305.01210>

- Merrill, W., & Sabharwal, A. (2023). The parallelism tradeoff: Limitations of log-precision transformers. *Transactions of the Association for Computational Linguistics*, 11, 531-545. https://doi.org/10.1162/tacl_a_00562
- METR. (2025, 10 de julio). *Measuring the impact of early-2025 AI on experienced open-source developer productivity*. <https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/>
- Naur, P. (1985). Programming as theory building. *Microprocessing and Microprogramming*, 15(5), 253-261. [https://doi.org/10.1016/0165-6074\(85\)90032-8](https://doi.org/10.1016/0165-6074(85)90032-8)
- Naur, P., & Randell, B. (Eds.). (1969). *Software engineering: Report on a conference sponsored by the NATO Science Committee, Garmisch, Germany, 7th to 11th October 1968*. Scientific Affairs Division, NATO.
- Osmani, A. (2024, 4 de diciembre). *The 70% problem: Hard truths about AI-assisted coding*. <https://addyo.substack.com/p/the-70-problem-hard-truths-about>
- Pearce, H., Ahmad, B., Tan, B., Dolan-Gavitt, B., & Karri, R. (2022). Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions. En *2022 IEEE Symposium on Security and Privacy (SP)* (pp. 754-768). IEEE. <https://doi.org/10.1109/SP46214.2022.9833571>
- Peng, S., Kalliamvakou, E., Cihon, P., & Demirel, M. (2023). *The impact of AI on developer productivity: Evidence from GitHub Copilot* (arXiv:2302.06590). arXiv. <https://arxiv.org/abs/2302.06590>
- Perry, N., Srivastava, M., Kumar, D., & Boneh, D. (2023). Do users write more insecure code with AI assistants? En *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS '23)* (pp. 2785-2799). <https://doi.org/10.1145/3576915.3623157>
- Rice, H. G. (1953). Classes of recursively enumerable sets and their decision problems. *Transactions of the American Mathematical Society*, 74(2), 358-366. <https://doi.org/10.1090/S0002-9947-1953-0053041-6>
- The San Francisco Standard. (2025, 27 de febrero). *Marc Benioff: Salesforce will hire no engineers in 2025 due to AI*. <https://sfstandard.com/2025/02/27/salesforce-marcbenioff-layoffs-tech-agents/>
- Shen, J. H., & Tamkin, A. (2026). *How AI impacts skill formation* (arXiv:2601.20245). arXiv. <https://arxiv.org/abs/2601.20245>
- Spracklen, J., Wijewickrama, R., Sakib, A. H. M. N., Maiti, A., Viswanath, B., & Jadliwala, M. (2025). We have a package for you! A comprehensive analysis of package hallucinations by code-generating LLMs. En *Proceedings of the 34th USENIX Security Symposium*. USENIX Association. <https://arxiv.org/abs/2406.10279>
- Stack Overflow. (2025). *2025 developer survey: AI*. <https://survey.stackoverflow.co/2025/ai/>
- Stanford HAI. (2026). *Artificial intelligence index report 2026*. Stanford Institute for Human-Centered Artificial Intelligence. <https://hai.stanford.edu/ai-index>
- Stroustrup, B. (2025, 9 de mayo). *Interview: Bjarne Stroustrup on 21st century C++, AI risks, and why the language is hard to replace* (T. Anderson, entrevistador). DevClass. <https://devclass.com/2025/05/09/interview-bjarne-stroustrup-on-21st-century-c-ai-risks-and-why-the-language-is-hard-to-replace/>
- Thompson, B. (2025, 20 de marzo). *An interview with OpenAI CEO Sam Altman about building a consumer tech company*. Stratechery. <https://stratechery.com/2025/an-interview-with-openai-ceo-sam-altman-about-building-a-consumer-tech-company/>

- Turing, A. M. (1936). On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, s2-42(1), 230-265.
<https://doi.org/10.1112/plms/s2-42.1.230>
- VandeHei, J., & Allen, M. (2025, 28 de mayo). *Behind the curtain: A white-collar bloodbath*. Axios.
<https://www.axios.com/2025/05/28/ai-jobs-white-collar-unemployment-anthropic>
- Veracode. (2025). *2025 GenAI code security report*.
<https://www.veracode.com/resources/analyst-reports/2025-genai-code-security-report/>
- Washizaki, H. (Ed.). (2024). *Guide to the Software Engineering Body of Knowledge (SWEBOK Guide), Version 4.0*. IEEE Computer Society.
<https://www.computer.org/education/bodies-of-knowledge/software-engineering>
- Welsh, M. (2023). The end of programming. *Communications of the ACM*, 66(1), 34-35.
<https://doi.org/10.1145/3570220>
- World Economic Forum. (2025). *The future of jobs report 2025*.
<https://www.weforum.org/publications/the-future-of-jobs-report-2025/>
- Xu, Z., Jain, S., & Kankanhalli, M. (2024). *Hallucination is inevitable: An innate limitation of large language models* (arXiv:2401.11817). arXiv. <https://arxiv.org/abs/2401.11817>
- Zeff, M. (2025, 29 de abril). *Microsoft CEO says up to 30% of the company's code was written by AI*. TechCrunch. <https://techcrunch.com/2025/04/29/microsoft-ceo-says-up-to-30-of-the-companys-code-was-written-by-ai/>